It Worked on My Machine

Repeatability, Reproducibility, Containers, and Evidence

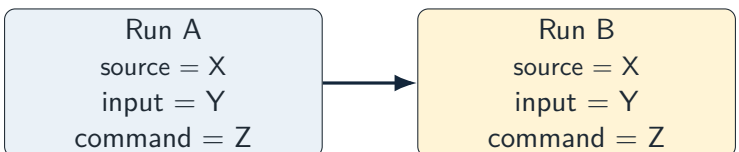
Week 3 question

If the code stays the same, what else can change?

SWOSU Computing Commons ▪ shared by DSCT and Computer Architecture

Start with a prediction

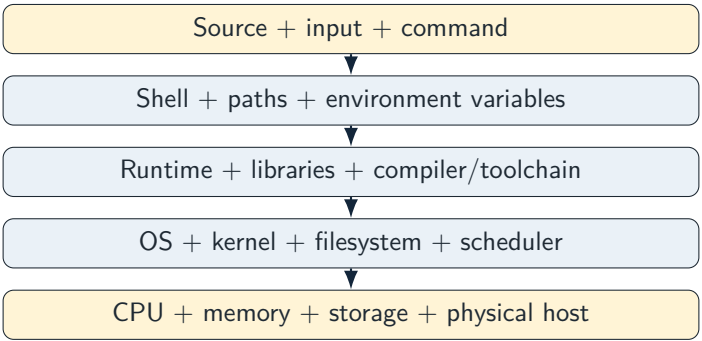
Same code. Same input. Same command. Same experiment?



Before we measure

Write down your prediction: will every observation be identical? Name three pieces of environment context that could matter.

Code is only one layer of the experiment



The Week 3 problem

Variation can sneak in at every layer. A useful experiment must say what we held fixed and what we merely observed.

Where can variation sneak in?

Software-side variation

- different shell or path rules
- different Python / Java / compiler
- different package versions
- different environment variables
- different filesystem contents

Host-side variation

- different CPU architecture
- different memory available
- different kernel behavior
- different machine load
- different timing and storage behavior

Key idea

"It worked on my machine" is not an excuse. It is a clue that the experiment had hidden context.

Control it or record it

You cannot freeze the universe. So what do you do?

CONTROL IT

Hold a variable fixed when the experiment reasonably can.

- source revision
- input data
- tool versions
- entry command

RECORD IT

Preserve context when it cannot or should not be fixed.

- executing architecture
- kernel / OS identity
- timestamp
- visible resources

A reproducibility contract

What must another person know to repeat the reasoning task?

Identity	source revision, image identity, or course snapshot
Input	the exact file/data used
Action	exact command or wrapper
Scope	host-visible, WSL-visible, container-visible, etc.
Evidence	output shape, receipt, artifact, or measurement
Boundary	limitations another person must know

Contract, not incantation

You do not need to memorize every hidden command. You do need to know what the wrapper promises and what evidence it leaves behind.

A receipt is evidence that survives the terminal

A useful receipt can preserve:

- source/tool identity
- execution scope
- relevant environment facts
- input/output identity
- timestamp or run identity
- pass/fail plus a reason

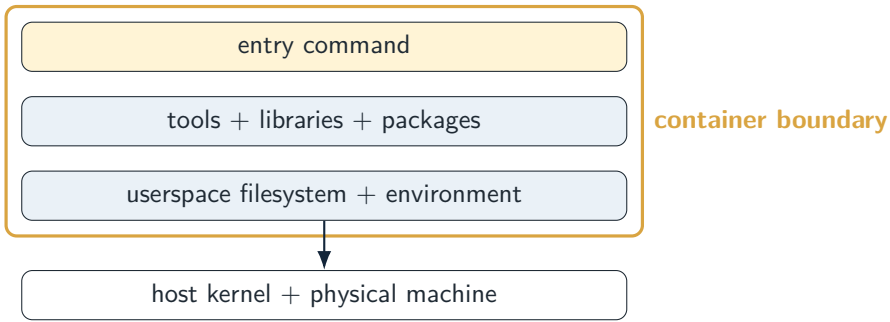
Do not overclaim

A receipt records what happened. It does not automatically prove **why** it happened.

Observation $\neq$ explanation

So what is a container?

Could we put part of the experiment in a controlled box?



Our Week 3 reason

We use a container so we can hold more of the **software environment** constant while deliberately observing what still depends on the host.

Image vs. running container

IMAGE

A frozen template that describes the environment we want.

- tools
- packages
- filesystem
- default command

CONTAINER

A running instance created from that image.

- starts
- performs work
- can be discarded
- does not erase the image

Vocabulary worth keeping

Image = template. Container = disposable running instance.

Pinned identity beats “latest”

What happens if the environment silently changes next week?

Floating tag

`:latest`

- easy to type
- may point somewhere new later
- weak evidence for reproduction

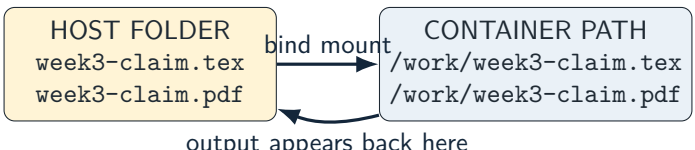
Pinned identity

Digest / fixed version

- identifies one exact image
- does not silently drift
- stronger reproducibility contract

Your files can cross the boundary

How does your work get into the container and back out?



One file, two path names

The file lives on the host, but the container sees that mounted folder at its own path. Confusing host paths and container paths is a normal, diagnosable mistake.

What containers help control

Often controlled well

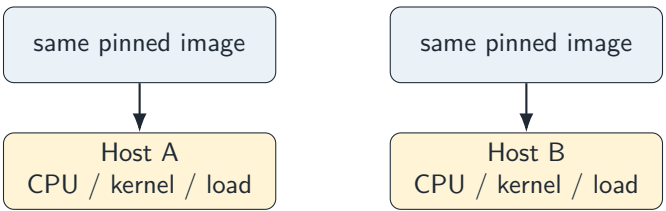
- userspace filesystem
- installed packages
- tool versions
- environment variables
- entry command
- known working directory

Still inherited / variable

- physical CPU
- host kernel
- host load and scheduling
- storage hardware
- timing noise
- every possible platform quirk

Containers do not make machines identical

Two containers can share an image and still observe different hosts.



Bad claim

"The container makes both computers the same."

Better claim

"The container fixes more of the software environment, making remaining host-dependent differences easier to identify."

The known-good run

First prove the supplied environment can do one tiny job.

1. Start with the course-provided source.
2. Run the course-provided wrapper.
3. Read the image, host path, container path, source, output, and result lines.
4. Open the produced artifact.
5. Confirm the artifact actually reflects the source you ran.

For DSCT

The tiny job is a `.tex` claim/proof compiled into a PDF. The wrapper handles the container command; your job is to understand the contract and verify the artifact.

Then perturb it on purpose

A system you can only use when nothing goes wrong is not yet a tool.

PERTURB → DIAGNOSE → PROPOSE → RERUN → VERIFY

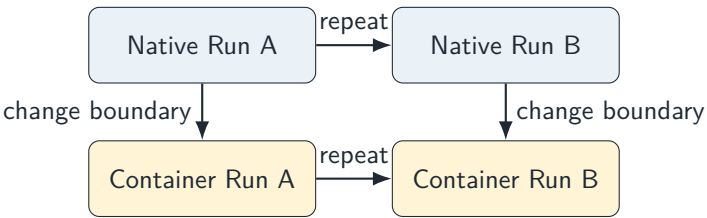
Common bounded failures:

- wrong host path
- missing source file
- runtime/image problem
- content/tool error

Evidence habits:

- read the reason first
- classify before changing things
- AI suggestions are proposals
- rerun after the repair
- inspect the real artifact

Native vs. container is an experiment



Questions worth asking

What stays stable? What changes? Which changes are expected? Which differences matter to the claim you are trying to make?

Repeatability and reproducibility are not the same claim

REPEATABILITY

Can I perform the same procedure again in the same environment and obtain the required evidence shape?

REPRODUCIBILITY

Can another environment preserve the important contract well enough to reproduce the reasoning or result?

Neither means

"Every bit of both machines was identical."

Make only the claim your evidence can carry

A bounded claim

The same _____ and _____ produced _____ across repeated runs. The receipt describes _____, but it does not prove _____.

Ask yourself

- What did I actually observe?
- What did I control?
- What did I merely record?

Then state

- one supported conclusion
- one limitation
- one thing more evidence would decide

One lesson, two course lenses

	DSCT	Computer Architecture
Main question	What claim can the evidence justify?	What environment/machine layer changed?
Receipt	What does it prove and not prove?	What did the process actually observe?
Container	How much confidence does the boundary buy?	Which layers moved inside/outside the boundary?
Difference	Is it relevant to the argument?	Why might the value legitimately differ?

Your Week 3 mission

1. **Predict** what environment context might matter.
2. **Run** a known-good task and preserve a receipt.
3. **Inspect** the container boundary and the artifact it produced.
4. **Perturb** one bounded thing and recover from the failure.
5. **Compare** what stayed stable and what changed.
6. **Defend** one claim that your evidence genuinely supports.

The point is not container syntax

The point is learning to make a computing result easier to reproduce, inspect, diagnose, and defend.

Same code is not the same experiment.

Control what you can.

Record what you cannot.

Make only the claim your evidence can carry.